
- al programa SQL le da igual si escribes la sentencia en mayúsculas o minúsculas. Se interpretará correctamente.
- Es importante escribir de forma correcta el nombre de las tablas con las que interactuemos.

CLÁUSULAS SQL Y OPERADORES

CLÁUSULAS

Principalmente encontraremos **5 cláusulas en SQL**:

1. **FROM**: nos indica las tablas de las cuales se quieren obtener registros
2. **WHERE**: especifica las condiciones o criterios de los registros seleccionados
3. **GROUP BY**: agrupa los registros seleccionados en función de un campo
4. **HAVING**: especifica condiciones o criterios que deberán cumplir los grupos
5. **ORDER BY**: ordena registros seleccionados en función de un campo

PRIORIDADES DE CLÁUSULAS



Comentario: es muy raro que se den todas las cláusulas en una única instrucción SQL, pero en el caso de que se diera este sería el orden.

OPERADORES

OPERADORES DE COMPARACIÓN

Operador	Significado
<	Menor que
>	Mayor que
=	Igual que
>=	Mayor o igual que
<=	Menor o igual que
<>	Distinto que
BETWEEN	Entre. Utilizado para especificar rangos de valores
LIKE	Cómo. Utilizado con caracteres comodín (? *)
In	En. Para especificar registros en un campo en concreto

OPERADORES EXTRAS:

- **ilike**: tiene la misma función que **like** pero no diferencia entre mayúsculas y minúsculas.

ANOTACIÓN: Al utilizar los operadores de comparación, hay que tener cuidado con los tipos de texto que importemos, ya que muchas veces al pasar una lista directamente puede que si no revisamos los **tipos de datos**, ciertas características como fechas, números... se pasen como tipo texto.

OPERADORES DE LÓGICOS

Operador	Significado
AND	Y lógico
OR	O lógico
NOT	Negación lógica

ANOTACIÓN: al utilizar los operadores lógicos muchas veces nos pedirán consultas del estilo “se solicita ver los productos de la sección **deporte y cerámica**”, aquí el lenguaje natural tendería a utilizar el operador “**AND**”, no obstante hay que mirar a la sentencia desde una perspectiva de **lenguaje estricto**, no hay ningún artículo que pertenezca a dos secciones a la vez.

CARACTERÍSTICAS SQL:

- los datos de tipo **texto** deberán ir entre comillas.
- Para seleccionar todos los campos podemos utilizar el **asterisco** “*”.
- Las **fechas** en SQL van entre comillas simples.
- Cada gestor de datos tiene su propia versión de SQL, **revisar postgresSQL**

CARACTERÍSTICAS SQL:

- En postgresSQL nos permite un atajo de agregar todas las tablas al incluir el **asterisco**, y luego incluir una tabla para modificarla. *EJEMPLO:*
→ **select *, salary*14 as “Salario_anual”**
- Si renombramos una tabla con otro nombre al usar el “**ORDER BY**” podemos indicarle el nombre que le customizamos.

CLÁUSULA ORDER BY

Por regla general vamos a encontrar que las tablas se van a ordenar en base al atributo primario.

CÓMO ORDENAMOS UNA CONSULTA

Para ello haremos uso de la cláusula **ORDER BY**. Esta cláusula nos permitirá tanto ordenar para numéricos como para alfanuméricos. En el caso de solo utilizar la sentencia “**ORDER BY Sección**” en el caso de que nos encontremos con letra lo hará por defecto **alfabéticamente** y en el caso de los números lo hará de **menor a mayor**.

Como modificar el ordenado → eso lo lograremos mediante una instrucción “**DESC**”, con él logramos que los resultados sean descendientes. Para ascendente **ASC**

¿Se puede concatenar el ordenamiento?

Si, esto se puede hacer en la sentencia SQL, para ello haremos uso de la coma al escribir la instrucción en la cláusula ORDER BY, ejemplo: “**ORDER BY Sección, Precio**”, tanto seccion como precio son dos atributos de nuestra tabla pero la hemos concatenado para que ambas aparezcan ordenadas. (En el caso de querer ordenarlas ascendente o descendente colocar la instrucción tras el atributo)

Hay un peros, debemos comprender que la siguiente concatenación significa que primero dentro del atributo **sección** lo ordena alfabéticamente y una vez ordenada, dentro de **cada** atributo **apartado** lo ordenara por precios.

CONSULTAS DE AGRUPACIÓN

Estas consultas se encargan de realizar cálculos por grupos, cogen registros de una tabla, los ordenan con base a un criterio y con ellos realizan un cálculo.

Para llevar a cabo las consultas de agrupaciones haremos uso de las “**funciones de agregado**”. Algunas de ellas son:

Función	Descripción
AVG	Calcula el promedio de un campo
COUNT	Cuenta los registros de un campo
SUM	Suma los valores de un campo
MAX	Devuelve el máximo de un campo
MIN	Devuelve el mínimo de un campo

Para llevar a cabo una consulta de agrupación necesitaremos de dos campos:

1. Campo de agrupación: que hace referencia al nombre del apartado donde nos encontremos un ejemplo sería una sección.
2. Campo de cálculo: donde encontraremos los datos numéricos con los que queremos interactuar, un ejemplo serían los precios.

ANOTACIÓN: cuando estemos utilizando la función **count** debemos tener cuidado con los campos en blancos ya que si utilizamos la misma sobre un campo con información vacía esta no la va a contar.

ROUND BY: Es una **función** que se aplica a valores numéricos y tiene como función redondear números a una cantidad de decimales.

Ejemplo: `round(AVG(precipitacion_total))`

Resultado:

Sevilla	0.00
---------	------

EJEMPLO: `select Sección, sum(Precios) from Producto GROUP BY Sección;`

SECCIÓN	SUM(PRECIO)
CERÁMICA	442.3498
CONFECCIÓN	501544.2935
DEPORTES	1305.8123
FERRETERÍA	95.3806
JUGUETERÍA	2516.7141
OFICINA	39.7606

Si queremos ordenar el campo de los precios podemos incluir un **ALIAS** al campo `sum(Precio)`, ejemplo: `select Sección, sum(Precio) as Suma_Articulos from Productos`

group by Sección order by Suma_Articulos;

Cuando utilizamos “**GROUP BY**” la finalidad es obtener un resultado por cada grupo, sin la cláusula nos saldría un **solo** resultado para toda la tabla.

Principalmente en estos tipos de consultas actuar solo con dos tipos de campos, más campos pueden traer problemas y según qué gestor pueden influir de una manera u otra.

ANOTACIÓN: para las consultas de agrupación vamos a sustituir la cláusula **WHERE** por la cláusula **HAVING**. Este es un concepto importante ya que **WHERE** nos permite filtrar las filas antes de agrupar, mientras que **HAVING** nos permite filtrar los grupos **después** de agrupar.

CASE WHEN & ELSE SQL

Bien, nos introducimos en el apartado del “**if & else**” pero en SQL, hay que distinguir y es que esto es una **expresión**.

¿Qué es una expresión? → Es cualquier cosa que produce un valor. Y es un concepto que no hemos tocado hasta la fecha actual, y no solo esto es una expresión sino que los cálculos, los textos que introducimos a cascaporras... pueden ser expresiones dado que cumplen con la función establecida.

Y BIEN, COMO FUNCIONA ESTA EXPRESIÓN?

Dentro de esta expresión vamos a encontrar palabras que nos van a servir para establecer las condiciones:

- **CASE (inicio):** es el comienzo de la lógica
- **WHEN:** es cuando la condición es evaluada
- **THEN:** es el resultado de la lógica si es cierta
- **ELSE:** es **opcional** devolver un valor, esta condición se cumplirá cuando la lógica sea falsa.
- **END (final):** Final de la lógica

Ejemplo de una expresión lógica →

```
CASE
  WHEN Sales > 50 THEN 'High'
  WHEN Sales > 20 THEN 'Medium'
  ELSE 'Low'
END
```

REGLA OBLIGATORIA: cuando estemos usando bloques de lógica, tanto el **THEN** como el **ELSE** deberán devolver el mismo tipo de dato (int, string, double...).

ANOTAR: es importante entender la lógica de aquí, y es que dentro de estos bloques de **lógica** no vamos a poder introducir alias.

ANOTAR 2: estos bloques de lógica los vamos a poder **utilizar** en prácticamente cualquier cláusula.

TEMA 5: SQL (JOINS & FECHAS)

FECHAS & HORAS

APARTADO PRÁCTICA

(En los apartados de práctica incluir ejercicios y ejemplos que para reforzar el aprendizaje)

EJERCICIO BOLETIN CLASE:

/*Seleccionar aquellos empleados cuyo número de departamento esté entre el 8 y el 12 (ambos excluidos). */

```
select *  
from employees  
where department_id > 8  
      and department_id < 12  
;
```

Anotación: se podría utilizar el **between** en este caso, sin embargo sería más correcto en esta situación.

/*Seleccionar todos los países que contengan una letra a en la segunda posición.*/

```
select *  
from countries  
where country_name ilike '_a%'  
;
```

Anotación: utilizamos el operador "**ilike**" ya que nos permite incluir en el rango tanto mayúsculas como minúsculas.

/*Seleccionar el nombre y apellidos de los empleados cuyo salario bruto anual sea mayor o igual que 100000\$, y que no sean contables (job_id = 6). */

```
select first_name, last_name  
from employees  
where salary*12 >= 100000 /* El 12 equivale a los meses de año */
```

```
and employee_id != 6  
;
```

/*Seleccionar aquellos departamentos cuyo nombre contenga dos letras t.*/

```
select *  
from departments  
where department_name ilike '%t%t%'  
;
```

ANOTACIÓN: acordarse del ilike para filtrar tanto mayúsculas como minúsculas.

/*Seleccionar las localizaciones que estén en las ciudades de Toronto u Oxford, o bien en el estado de California.*/

```
select *  
from locations  
where (city in ('Toronto', 'Oxford'))  
      or (state_province in ('California'))  
;
```

/* Selecciona el total de hombres de las provincias de Extremadura en el año 2002 y 2003 (demografia_basica) */

```
select sum(hombres)  
from demografia_basica  
where anio in (2002, 2003)  
      and provincia in ('Badajoz', 'Cáceres')  
;
```

ANOTACIÓN: para llevar a cabo esto utilizaremos *funciones*, a día de hoy si utilizamos una función no podremos mostrar más tablas.

/* Selecciona, ordenando de mayor a menor aquellas provincias y años en los que hubo una población superior o igual a 600.000 personas o de hombres o de mujeres (demografia_basica). Como resultado se deben mostrar todas las columnas de cada fila. */

```
select *  
from demografia_basica  
where hombres >= 600000  
      or mujeres >= 600000  
order by provincia desc, anio desc  
;
```

ANOTACIÓN:

/* Acota la búsqueda anterior para que sea solamente de los últimos 5 años. */

```

select *
from demografia_basica
where hombres >= 60000
      or mujeres >= 60000)
      and anio >= 2010
order by greatest (hombres, mujeres) desc;

```

ANOTACION: actualmente no tenemos conocimientos como para acotar los últimos 5 años, así que tenemos dos opciones, indicarlo mediante el operador **and**.

ANÉCDOTA: el greatest no es necesario pero nos ayuda

/* Selecciona cuántas mujeres de entre 20 y 29 años vivían en Andalucía en el año 2018 (demografia_avanzada). Ordena la salida por número de habitantes (menor a mayor) NOTA: No debe aparecer el total, sino el desglose por provincia y rango de edad*/

```

select *
from demografia_avanzada
where sexo = 'M'

      and edad_menor >= 20

      and edad_mayor <= 29

      and anio = 2018

      and provincia in ('Jaen','Cordoba', 'Sevilla', 'Huelva',
                        'Cádiz', 'Málaga', 'Granada', 'Almería')

order by numero_habitantes asc;

```

/* Selecciona el número de hombres que vivían en la comunidad de Madrid entre los años 2010 y 2015 (ambos inclusive), donde el rango de edad sea menores de 20 años o mayores de 65 (demografia_avanzada). La salida se debe ordenar por anio (menor a mayor), número de habitantes (mayor a menor) y edad (menor a mayor) */

```

select *
from demografia_avanzada
where sexo = 'H' and provincia = 'Madrid'

      and anio BETWEEN 2010 and 2015

      and (edad_menor <=20

      or edad_mayor >= 65)

Order by anio ASC, numero_habitantes DESC, edad_menor asc

```

;

/*Selecciona la precipitación total media para todas las estaciones meteorológicas de la provincia de Sevilla para el mes de Abril de 2019. Redondea el resultado con 2 decimales */

```
select provincia, round(AVG(precipitacion_total), 2) as media_precipitacion
from climatologia
where provincia in ('Sevilla')
      and fecha >= '2019-04-01' and fecha < '2019-05-01' (INCORRECTA)
group by provincia
;
```

Filtrar fechas: nos recomienda que no nos fiemos de la transformación automática y pasemos la fecha a tipo texto → (fecha::text) y quedaría (and fecha::text like '2019-04')
En el próximo tema nos revelará una mejor forma de hacerlo.

ANOTACIÓN: cuando establezca un rango de fecha una buena práctica es en vez de tener que conocer los días que tiene un mes poner el rango desde que empieza hasta el siguiente mes.

/*Selecciona los datos meteorológicos para aquellas estaciones cuyo nombre contenga la letra E, en los últimos días del mes de Febrero, Marzo y Abril de 2019. Ordena los resultados por provincia ascendente, estación ascendente y fecha, descendente. */

```
select *
from climatologia
where estacion ilike '%e%'
      and (fecha::text ilike '2019-02-28'
            or fecha::text ilike '2019-03-31'
            or fecha::text ilike '2019-04-30')
      and fecha::text like '2019%'
      and substr (fecha::text, 6, 2)
            in ('02', '03','04')
      and substr (fecha::text,9) ::int >= 20
order by provincia asc, estacion asc, fecha desc
;
```

Otra forma de filtrarlos meses → [fecha::text in ('2019-02-28', '2019-03-31', '2019-04-30')]

ANOTACIÓN: Esta variante nos permite filtrar por los últimos días del mes.

ANOTACIÓN: lo primero que quiero comentar es preguntar si hay alguna forma de coger el día anterior a una fecha dada, de esa forma no habría que conocer el calendario y hay menos probabilidad de fallo. El otro aporte es que es necesario entender los paréntesis, en este caso me daba fallo al no colocar el doble paréntesis.

/* Selecciona la suma de la precipitación total acumulada en Galicia durante el mes de Septiembre de 2019 */

```

select provincia, sum(precipitacion_total) as suma_precipitacion
from climatologia
where provincia in ('A Coruña')
      and fecha >= '2019-09-01'
      and fecha < '2019-10-01'
group by provincia
;

```

ANOTACIÓN: recordar que cuando nos piden las provincias de un lugar no hay que escribir las comunidades autónomas. Otra cosa es acordarse del **group by**.

/*Selecciona el porcentaje de precipitación de cada tramo de horas (0 a 6, 6 a 12, 12 a 18, 18 a 24) con respecto de la precipitación total para las estaciones de Castilla La Mancha en la primavera de 2019 (21 Marzo a 20 de Junio). Como resultado de la consulta debe aparecer la fecha, estación, provincia, precipitación total y los porcentajes de cada tramo horario. Redondea los porcentajes con 2 decimales. */

```

=====
select fecha,
      estacion, provincia , precipitacion_total,
      round (
        case
          when precipitacion_total > 0 then
            (precipitacion_0_a_6/precipitacion_total*100)
          else (precipitacion_0_a_6*0)
        end
      , 2) as porc_precipitacion_0_a_6,

      round (
        case
          when precipitacion_total > 0 then
            (precipitacion_6_a_12/precipitacion_total*100)
          else (precipitacion_6_a_12*0)
        end
      , 2) as porc_precipitacion_6_a_12,

      round (
        case
          when precipitacion_total > 0 then
            (precipitacion_12_a_18/precipitacion_total*100)
          else (precipitacion_12_a_18*0)
        end
      , 2) as porc_precipitacion_12_a_18,

      round (
        case

```

```

        when precipitacion_total > 0 then
            (precipitacion_18_a_24/precipitacion_total*100)
        else (precipitacion_18_a_24*0)
    end
, 2) as porc_precipitacion_18_a_24

from climatologia
where provincia in ('Albacete' , 'Toledo')
    and fecha >= '2019-03-21' and fecha < '2019-06-21'
order by provincia
;
=====

```

ANOTACION: Cuando utilicemos expresiones de lógica debemos tener cuidado donde las colocamos, ya que al usarlas en la cláusula select al principio las colocaba antes que las columnas y no es así, la lógica va dentro del propio atributo, otra cosa es que dentro de la lógica no podemos devolver los **alias**.

EJERCICIO 7 - BOLETIN 1

/*EJERCICIO 1 */

```

select *
from vuelos
order by salida asc;

select* , (
    case
        when descuento is not null or descuento > 0
        then round((precio-(precio*descuento/100)),2)::text || '€'
        when descuento is null or descuento = 0
        then round(precio,2)::text || '€'
    end
) as Precio_con_descuento
from vuelos
where salida::text ilike ('2019-12%')
    and desde in ('Sevilla', 'Málaga')
    and hasta in ('Madrid', 'Barcelona')
    and precio >= 80
;

```

/* EJERCICIO 2 */

```

select *, upper(
    concat(
        substr(desde,1,3),
        '-',

```

```

        substr(hasta,1,3),
        '-',
        case
            when id < 10 then concat('000',id::text)
            when id < 100 then concat('00',id::text)
            when id < 1000 then concat('0',id::text)
            else id::text
        end
    )
) as codigo_ruta
from vuelos
where salida::text ilike '2020-12%'
    and desde in ('Sevilla', 'Málaga')
    and hasta in ('Madrid', 'Barcelona')
    and precio >= 8
;

```

/* EJERCICIO 3*/

```

select COUNT(*) AS "Número total de vuelos",
        MAX(precio) as precio_maximo ,MIN(precio) as precio_minimo,
        round(avg(precio),2) as precio_medio, round(avg(coalesce(descuento, 0)),2)
from vuelos
where salida::text ilike ('2020%')
    and hasta in ('Madrid', 'Barcelona', 'Sevilla', 'Málaga')
    and precio between 60 and 500
    and desde::text not like (hasta::text)
;

```

/*EJERCICIO 4*/

```

select *, (
    case
        WHEN descuento is null or descuento = 0 then 'Sin_Descuento'
        When descuento >= 30 then 'Descuento_Alto'
        else 'Descuento_Bajo'
    end
) as tipo_descuento
from vuelos
where salida::text ilike ('2020-03%')
    and desde in ('Londres', 'París')
    and hasta in ('Sevilla', 'Málaga')
    and precio between 60 and 300
    and descuento is not null or descuento > 0
    and precio > 200
;

```

/*EJERCICIO 5 */

```

select id as identificador, upper(desde) as origen, upper(hasta) as destino,

```

```

precio, round(precio,-1) as Precio_Decena_Cercana
from vuelos
where salida::text ilike ('2019%')
      and (hasta ilike ('B%') or hasta ilike ('Á%'))
      and descuento >= 10
      and precio < 200
;

```

PRÁCTICA DE FECHAS

```

/*Queremos mostrar dd/mm/yyyy, day, dd/mm/yyyy */
select to_char(current_date, 'DD/MM/YYYY');
select to_char (current_date, 'DAY, DD/MM/YYYY');

```

/*Selecciona la precipitación total media para todas las estaciones meteorológicas de la provincia de Sevilla para el mes de Abril de 2019. Redondea el resultado con 2 decimales */ Incluyendo los datos de fecha

```

select precipitacion_total, estacion, fecha
from climatologia
where fecha between make_date (2019,4,1)
      and make_date (2019,4,30)
      and provincia = 'Sevilla'
order by estacion, fecha;

```

/*Selecciona el porcentaje de precipitación de cada tramo de horas (0 a 6, 6 a 12, 12 a 18, 18 a 24) con respecto de la precipitación total para las estaciones de Castilla La Mancha en la primavera de 2019 (21 Marzo a 20 de Junio). Como resultado de la consulta debe aparecer la fecha, estación, provincia, precipitación total y los porcentajes de cada tramo horario. Redondea los porcentajes con 2 decimales. */

```

select ...
from climatologia
where fecha between make_date (2019,3,21) and make_date (2019,6,21)
      and provincia in ('Cuenca'...)
order by fecha;

```

-- EJERCICIO 1

```

select employees.first_name,employees.last_name
from employees join departments using (department_id)
where departments.department_name in ('IT','Finance')
      and extract(month from hire_date) in (4,6)
;

```

--EJERCICIO 2

```

select mana.first_name, mana.last_name
from employees emp join employees mana

```

```
        on emple.manager_id = mana.employee_id
        join departments d on emple.department_id = d.department_id
where d.department_name = 'Administration';
```

-- EJERCICIO 3

```
select countries.country_name
from departments join locations using (location_id)
        join countries using (country_id)
where departments.department_name = 'Public Relations'
;
```

--EJERCICIO 4

```
select employees.employee_id ,employees.first_name, employees.last_name,
regions.region_name
from employees join departments using (department_id)
        join locations using (location_id)
        join countries using (country_id)
        join regions using (region_id)
where regions.region_name = 'Americas'
;
```

--EJERCICIO 5

```
select *
from countries;
```

```
select dependents.first_name as Nombre_hijo, dependents.last_name as Apellido_hijo,
        employees.first_name as Nombre_padre, employees.last_name as Apellido_padre,
regions.region_name as Region
from dependents join employees using (employee_id)
        join departments using (department_id)
        join locations using (location_id)
        join countries using (country_id)
        join regions using (region_id)
where regions.region_name = 'Americas'
order by countries.country_name
;
```

--EJERCICIO 6

```
select employees.first_name, employees.last_name, employees.hire_date, locations.city
from employees join departments using (department_id)
        join locations using (location_id)
where locations.state_province in ('Washington', 'California')
        and hire_date between make_date (1997,09,01) and make_date (1997,09,01)
;
```